

Jayden McKenna

CS380P

Lab 3: BST Comparison

Intro

This lab explores the performance of a Go based program that detects equivalent binary search trees across an input file. We compare the scalability and the parallelizability of detecting equivalent hashes as well as the true inorder equivalence. I implemented my program in three stages:

1. Build: construct the BSTs from the input
2. Hash Groups: computes the hashes for each BST and groups these identical hashes together
3. Compare: compare BSTs within the same hash group to see if they meet the equivalence criteria.

I implemented a parallelizable build function for the trees, but quickly realized that this was not needed for the assignment. I decided to leave this since it was already constructed and did not alter my measurements since we only needed the hash times and compare times.

Implementations

I provided two parallel implementations to the hashing function and compare function as well as a sequential implementation of each. Each version provides a deterministic and correct solution.

The sequential implementation acts as the baseline to compare the correctness of the parallel implementations. It also acts as a performance reference, as in any parallel program the goal is for the speedup to be greater than one.

Go provides two mechanisms for concurrency through the use of Go routines.

1. Channel based approach: Workers receive tasks through channels and send results back to a collector go routine. This is a much simpler model as it avoids any explicit locking and synchronization. Go channels have an implicit communication synchronization that makes it a lot easier on the programmer. The main drawback from using this approach is the scheduler incurs a lot of work and the implicit communication creates a lot of overhead.
2. Mutex based approach: A shared map is protected by a lock while multiple workers access it concurrently. The idea is that this reduces the amount of goroutines that are created which also reduces the amount of message passing (overhead). The main drawback we see here is the lock contention increases as the number of threads increase.

The arguments passed determines which of these implementations is used.

Step 2 – Hash Function:

- `data-workers = 1 && hash-workers = 1`: compute the hashes sequentially. A single worker computes hashes and groups the BSTs according to the hashes.
- `data-workers = 1 && hash-workers = i`: This calls the channel approach described above. The main goroutine distributes BSTs to a channel, while `i` workers concurrently compute hashes and return these results to the collector.
- `data-workers = i && hash-workers = i`: This calls the mutex approach. Each worker computes the hash values independently and then shares their answer to a map (protected by the mutex).

To switch between the compare function implementation:

- `comp-workers = 1`: Compares trees sequentially. A single goroutine does all the work. (as specified in the instructions document)
- `comp-workers = i`: This implementation spawns `i` goroutines to conduct comparisons and are distributed through a shared work queue. This bounds the number of active goroutines and scales well at moderate thread counts.
- `**comp-workers = 0**`: This is for the initial step 3 implementation. This spawns one goroutine per comparison check. This makes it simple as it only has one task to do and writes the result back to the adjacency matrix.

I use `comp-workers = 0` to signify the one per implementation arbitrarily, as the number of comp workers doesn't actually matter for this implementation. It is more so if the graders would want to test this manually. Otherwise, only the mutex is ran since it actually uses the comp-workers.

Theoretical Performance

We know the expected performance of a parallel program is largely limited to the points where the code must remain sequential. In addition, the synchronization and scheduling costs also bring down parallel efficiency. This gives us insight on what we might expect when running the program.

Hashing Implementations:

For the coarse dataset, hashing dominates the runtime. Since each tree can be processed independently, we should expect a near linear speedup when thread counts remain low. When the number of hash workers increases we can expect for the speedup to drop off in the channel implementation due to increasing communication overhead. For the mutex implementation, we should see a stark decline as thread counts go high because of lock contention when trying to access the shared map. In total, there should be a happy medium where the number of threads does give a deterministic speedup.

For the fine dataset, performance is expected to be much worse by design. This input intentionally introduces higher contention and synchronization which causes the contention and synchronization to dominate. Not the actual hash computations. It is plausible that at no point in will the parallel implementation be faster than the sequential implementation.

Comparison Implementations:

For the coarse dataset, the number of comparisons per hash group is relatively moderate. Since it has less to work with, we should see the implementation that spawns i go routines as a thread pool outperform the per comparison implementation. This is because the per comparison would slow down due to the creation of the many goroutines when its simply not necessary.

[I will reference the implementation that is dictated by the number of comp-workers as thread pool going forward.]

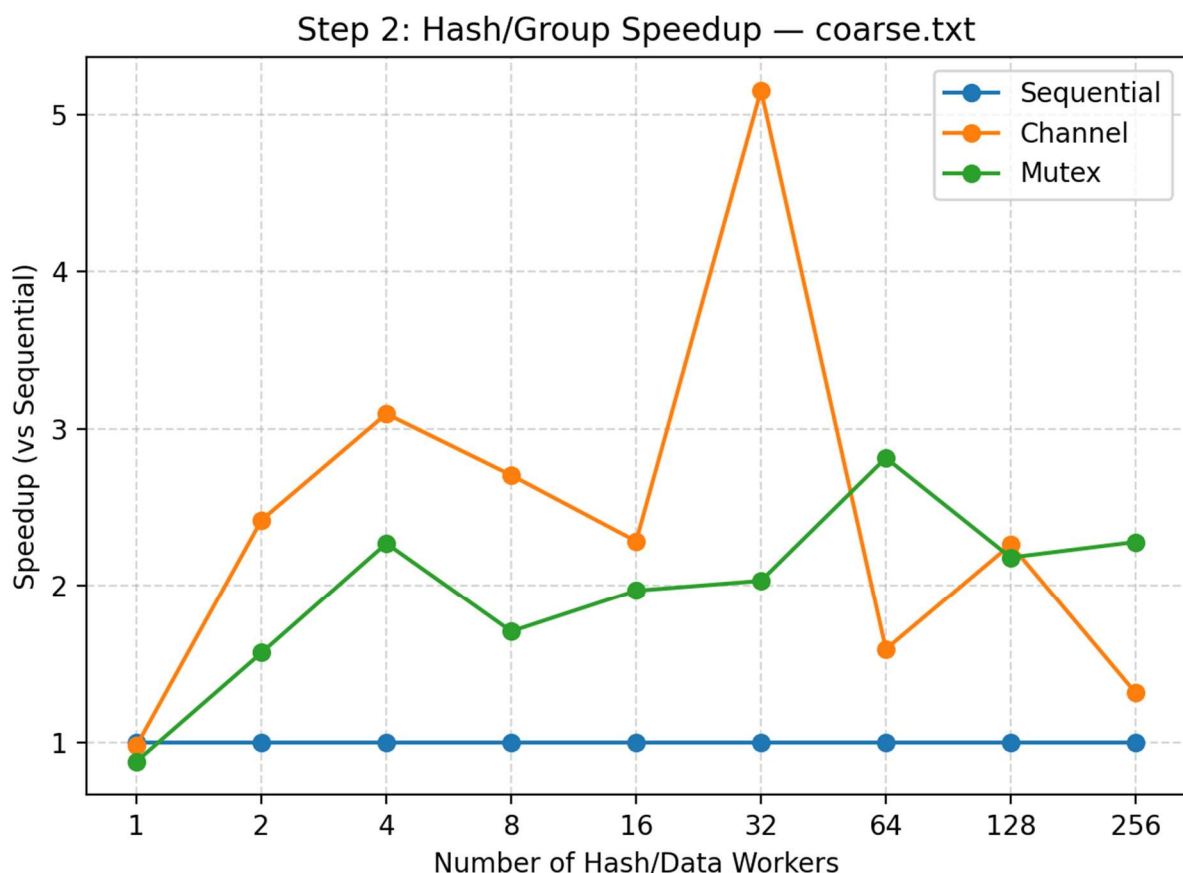
For the fine dataset, the performance difference becomes extreme. The thread pool should experience very bad speedup as the enormous input bottlenecks the access to the adjacency matrix.

Results

To test my implementations, I ran the program with the different number of workers 7 times and took the average elapsed time of each type of run.

Exploring the results from the Hashing implementations we get two interesting results.

Coarse Dataset



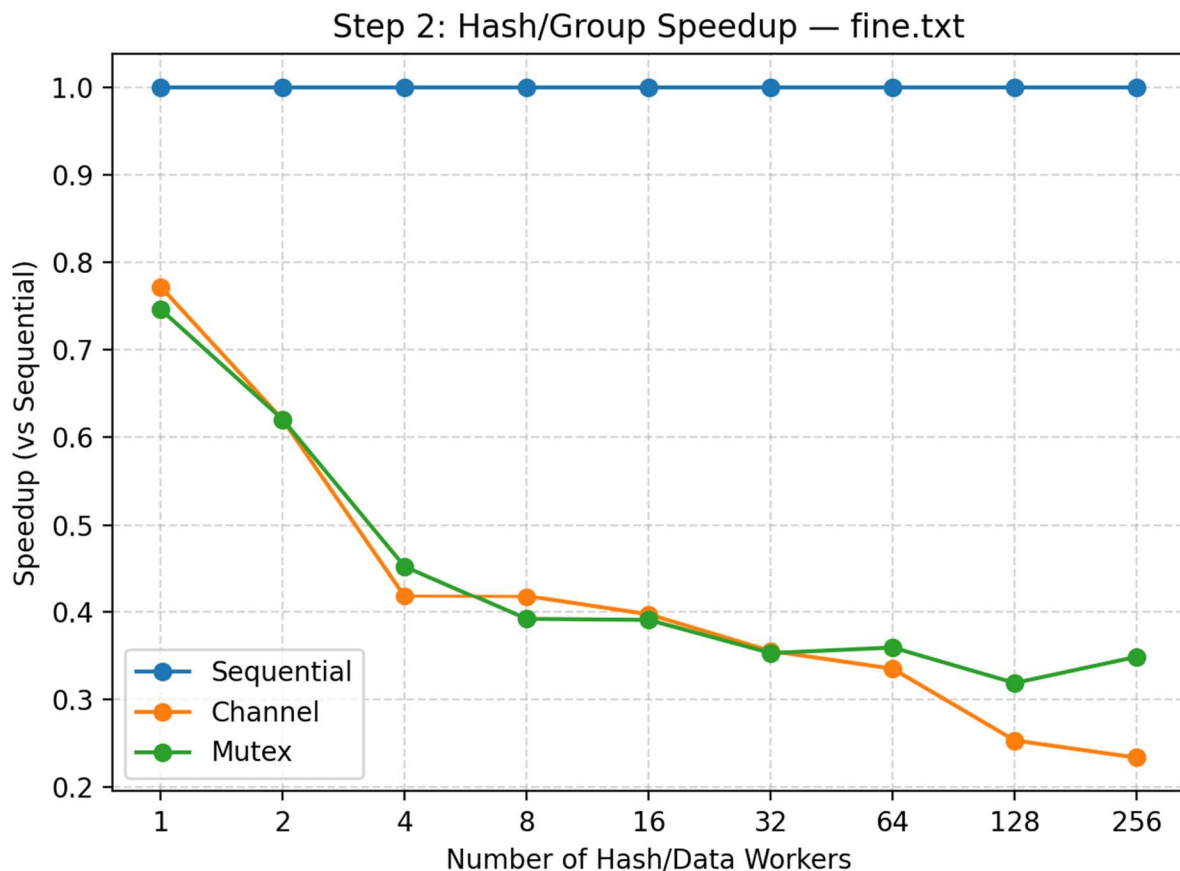
For coarse.txt, both the channel- and mutex-based implementations achieved significant speedup over the sequential baseline.

The channel implementation achieved the highest observed speedup of approximately 5x near 32 workers, while the mutex implementation peaked around 2.5–3x near 64 workers.

After these points, both began to taper off as overhead dominated the parallel benefit.

These results make sense: at low to moderate thread counts, hashing operations are largely independent, allowing nearly linear scaling. As thread count increases, channel scheduling and communication overhead begins to outweigh the parallel gain, and lock contention limits the mutex version.

Fine Dataset



For fine.txt, both parallel implementations performed worse than the sequential baseline.

Speedup fell steadily below 1x, reaching as low as 0.3–0.4x at higher thread counts. This is expected: the fine dataset was intentionally designed to contain many small, interdependent hash operations that emphasize synchronization costs. Since each hash operation completes quickly,

communication and locking dominate execution time, leaving little opportunity for true parallel speedup.

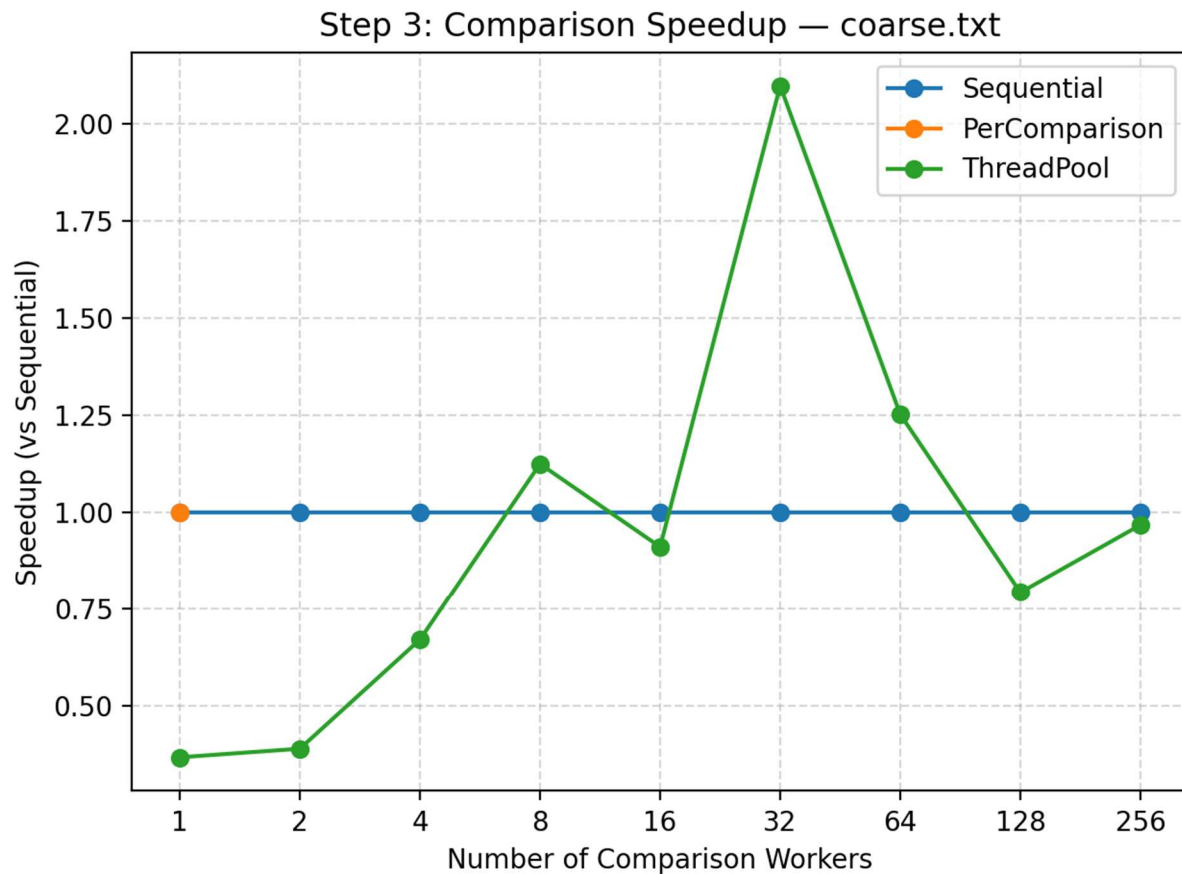
Step2 Q&A

- Which implementation is faster (and by how much)?
 - We see that the channel version was much faster despite it being much simpler to implement and requires little programmer implementation to handle the parallelism.
- Can Go manage goroutines well enough that you don't have to worry about how many threads to spawn anymore?
 - Not entirely. Go's scheduler is efficient and works in the short term, but as more goroutines are spawned the performance tanks as the synchronization costs increase.
- Which approach has more overhead?
 - The channel implementation introduces more overhead since communication and synchronization happen on each goroutine.
- How much faster are they compared to a single thread?
 - On the coarse dataset both implementations are much faster than spawning a singular thread, but when we get to the fine dataset, the sequential is actually the fastest implementation.
- Which approach do you find simpler?
 - The channel implementation is much simpler as it feels like I'm writing the normal sequential algorithm since I do not have to think about the critical zones and protecting any mutable states.

The hashing implementations explored the importance of granularity, explaining how fine-grained tasks experience diminishing returns much faster than coarse tasks.

Now let's look at the collected data from the comparison implementations. Where we will mainly focus on how the number of compare-workers affects the performance.

Coarse Dataset



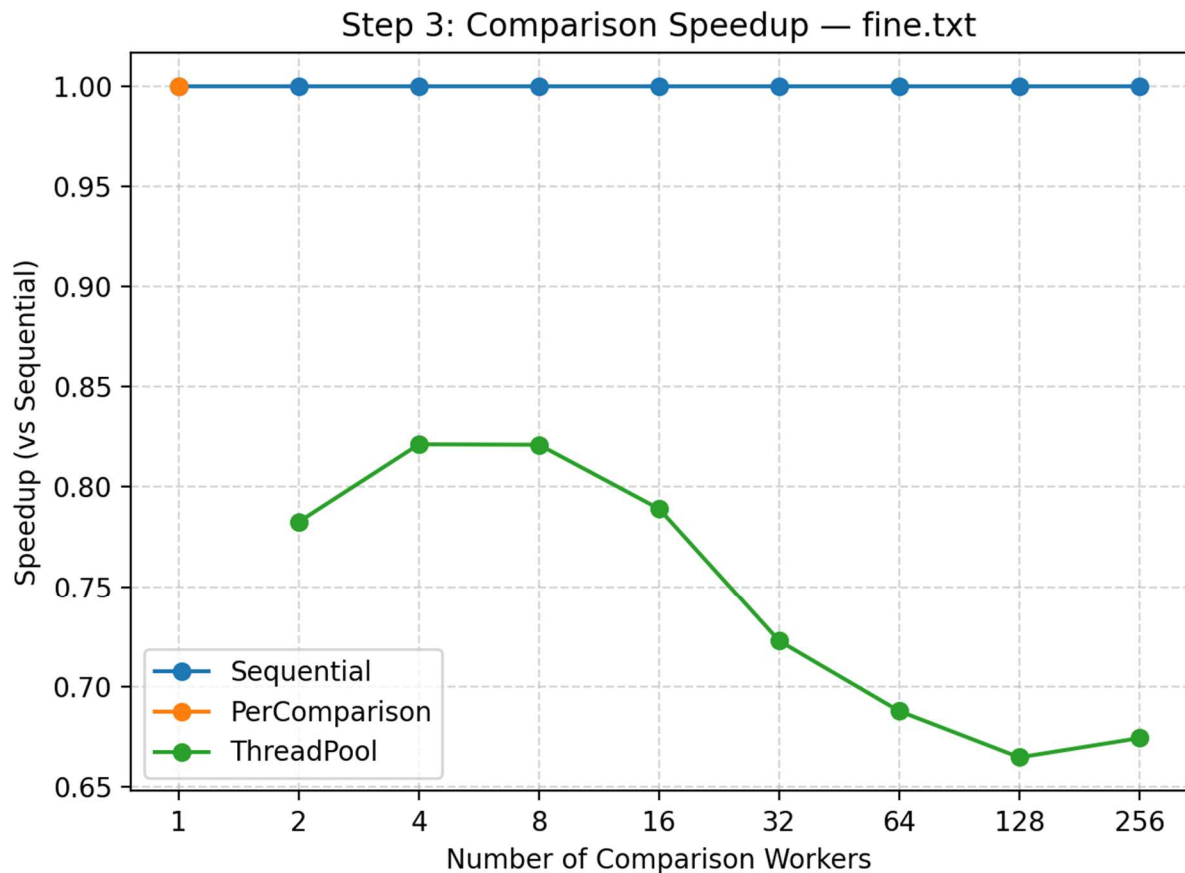
Coarse Dataset

For coarse.txt, the thread-pool implementation demonstrated clear performance improvements over the sequential baseline when given an optimal number of workers. Speedup peaked around 2x at 32 workers before tapering off at higher thread counts. This result reflects efficient task reuse within the pool. Once an optimal number of workers is reached, additional goroutines add scheduling and synchronization overhead without improving throughput.

The thread-pool approach balances concurrency and control. It prevents the overhead of launching thousands of short-lived goroutines. The performance curve follows the theoretical performance on what we might expect.

Fine Dataset

In the instructions, we were told the graphing of fine.txt was not needed as the large adjacency matrix would slow the program exponentially. I decided to test to see if this was true. This part is largely for my own experimentation and curiosity.



For fine.txt, the thread-pool implementation did not achieve meaningful speedup. Its performance plateaued below 1x speedup (around 0.7–0.8x of the sequential baseline) as thread count increased.

This decline is expected: fine-grained workloads contain an enormous number of tiny comparison tasks, so synchronization and queue contention dominate the total runtime. The fixed worker pool limits cannot overcome the high coordination costs of this dataset. This further justifies our conclusion we came to in step 2 about granularity, where the finer-grained tasks do not experience the benefits of parallelism compared to coarse tasks.

Step 3 Q&A

- How do the performance and complexity of both approaches compare with each other?

- The thread-pool approach performs significantly better than the per-comparison/sequential implementation. It is somewhat more complex due to queue synchronization and worker management, but it seems to be worth it as it scales efficiently to some optimal number of workers.
- How do they scale compared to a single thread?
 - The thread-pool version reaches roughly 2x speedup, while the per-comparison model provides no improvement over sequential execution.
 - The thread-pool implementation scales very poorly on fine.txt, but if we only compare the coarse tasks it does moderately well.
- Is the additional complexity of managing a thread pool worthwhile?
 - Yes. For coarse-grained workloads, the modest increase in implementation complexity yields deterministic and >1 speedup.
 - Inversely, for fine-grained tasks, the additional complexity is not worth it as the speedup < 1 .

TLDR Points

- Explored Go's concurrency mechanisms by parallelizing hashing and comparison of BSTs.
- On the coarse dataset, parallelization was effective in all implementation attempts.
 - The channel based hashing achieved a 5x speedup, and the mutex based hashing achieved a 3x speedup.
 - The thread pool comparison achieved a 2x speedup.
- On the fine dataset, speedup plummeted below the sequential versions for all implementation attempts. Excessive synchronization for fine-grained tasks proved to be less than optimal for parallelization.
- Overall, this lab taught me the importance of granularity in parallel tasks. The finer the tasks, the more overhead and bottleneck a parallel program has causing the speedup to be inefficient at best and devastating at worst.